

Dependency Injection - Cheat Sheet

Dependency Injection (DI) is the pattern of giving a class its dependencies from the outside. The class declares what it needs. Another component (often a framework) builds and hands over the actual instances.

It sounds small, but it's what makes Java code testable and swappable in any modern framework. Spring made DI mainstream in Java around 2003-2004. Today most frameworks (Spring, Guice, Dagger, Micronaut, Quarkus) revolve around it.

The Problem It Solves

Without DI, classes wire their own dependencies:

```
public class OrderService {
    private final EmailSender email = new SmtEmailSender();
    private final OrderRepository repo = new MySqlOrderRepository();

    public void place(Order o) {
        repo.save(o);
        email.send(o.customerEmail(), "order confirmed");
    }
}
```

Three problems:

1. **Hard to test.** You can't substitute a mock `EmailSender` without changing the code.
2. **Hard to swap.** Switching to a different repo means editing the service.
3. **Hard to compose.** Every class knows how to build every dependency it needs.

With DI:

```
public class OrderService {
    private final EmailSender email;
    private final OrderRepository repo;

    public OrderService(EmailSender email, OrderRepository repo) {
        this.email = email;
        this.repo = repo;
    }

    public void place(Order o) {
        repo.save(o);
        email.send(o.customerEmail(), "order confirmed");
    }
}
```

The service now declares what it needs, and someone else decides what to pass in. Tests pass in mocks. Production passes in real implementations. The service doesn't care.

DI vs Inversion of Control

People use these terms interchangeably, but they're slightly different.

- **Inversion of Control (IoC)** is the broader idea: the framework calls your code, rather than your code calling the framework.
- **Dependency Injection** is one specific way to achieve IoC. Your code declares what it needs in the constructor signature, and something else fills them in.

Other forms of IoC include event-driven callbacks, the Template Method pattern, and Service Locator. DI is the most popular flavor today.

The Three Forms of DI

1. Constructor Injection

Dependencies are required parameters of the constructor.

```
public class OrderService {
    private final EmailSender email;
    private final OrderRepository repo;

    public OrderService(EmailSender email, OrderRepository repo) {
        this.email = email;
        this.repo = repo;
    }
}
```

Pros:

- Dependencies are explicit and `final`.
- The object is fully constructed and valid after `new`.
- Easy to test by passing mocks directly.
- The compiler enforces required dependencies.

Cons:

- Long constructor signatures when a class has many dependencies. Usually a sign the class is doing too much.

This is the recommended form. Use it by default.

2. Setter Injection

Dependencies are set after construction.

```
public class OrderService {
    private EmailSender email;
    private OrderRepository repo;

    public void setEmail(EmailSender email) {
        this.email = email;
    }

    public void setRepo(OrderRepository repo) {
        this.repo = repo;
    }
}
```

Pros:

- Optional dependencies can stay null.
- Useful as an escape hatch for circular dependency hacks.

Cons:

- The object exists in a partially-constructed state.
- Dependencies can't be `final`.
- Easy to forget to call a setter.

Use only when a dependency is genuinely optional.

3. Field Injection

Dependencies are injected directly into fields via reflection. Requires a framework.

```
public class OrderService {
    @Autowired
    private EmailSender email;

    @Autowired
    private OrderRepository repo;
}
```

Pros:

- Less typing.
- Looks clean.

Cons:

- Fields can't be `final`.
- Hard to test without the framework, since you can't pass dependencies in directly.

- Hidden dependencies. You can't see what the class needs without scanning all its fields.
- Tied to the framework's annotations.

Avoid field injection in new code. The convenience isn't worth the testability cost.

Manual DI vs DI Container

You can do DI without any framework. Just wire dependencies yourself in `main` or in a factory:

```
public class Main {  
    public static void main(String[] args) {  
        EmailSender email = new SmtEmailSender();  
        OrderRepository repo = new MySqlOrderRepository(dataSource);  
        OrderService service = new OrderService(email, repo);  
  
        // use service  
    }  
}
```

For tiny apps, this is enough. As the dependency graph grows, hand-wiring everything becomes tedious and error-prone.

A **DI container** (Spring, Guice, Dagger) automates the wiring. You describe the dependencies, and the container builds the graph at startup.

Spring DI Example

The most common DI framework in Java. Configuration via annotations.

```

@Service
public class OrderService {

    private final EmailSender email;
    private final OrderRepository repo;

    public OrderService(EmailSender email, OrderRepository repo) {
        this.email = email;
        this.repo = repo;
    }

    public void place(Order o) {
        repo.save(o);
        email.send(o.customerEmail(), "order confirmed");
    }
}

@Component
public class SmtEmailSender implements EmailSender {
    public void send(String to, String subject) {
        // send email
    }
}

@Repository
public class MySQLOrderRepository implements OrderRepository {
    public void save(Order o) {
        // persist to MySQL
    }
}

```

Spring scans for `@Component` (and its specializations `@Service`, `@Repository`, `@Controller`) at startup, builds the graph, and injects via constructors. No XML, no boilerplate factories.

Bootstrap:

```

@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}

```

Common Annotations

Spring

Annotation	Meaning
<code>@Component</code>	Generic managed bean
<code>@Service</code>	Component carrying business logic
<code>@Repository</code>	Component handling data access
<code>@Controller</code> / <code>@RestController</code>	Web layer component
<code>@Autowired</code>	Inject by type (constructor, setter, or field)
<code>@Qualifier("name")</code>	Disambiguate when multiple beans match a type
<code>@Primary</code>	Mark a bean as the default when multiple exist
<code>@Value("\${prop}")</code>	Inject a property value
<code>@Configuration</code>	Class containing <code>@Bean</code> factory methods
<code>@Bean</code>	Explicit bean factory method
<code>@Scope("...")</code>	Set bean scope
<code>@Lazy</code>	Defer bean creation until first use

Jakarta (`jakarta.inject` / `javax.inject`)

Standard, framework-neutral. Works with Spring, Guice, Quarkus, and others.

Annotation	Meaning
<code>@Inject</code>	Equivalent to <code>@Autowired</code>
<code>@Named("name")</code>	Equivalent to <code>@Qualifier</code>
<code>@Singleton</code>	Singleton scope
<code>Provider<T></code>	Lazy or per-call instance lookup

Prefer Jakarta annotations when you want framework portability.

Bean Scopes

How long an instance lives.

Scope	Lifetime
singleton (default)	One instance per container, reused everywhere
prototype	New instance every time it's injected
request	One per HTTP request (web apps)
session	One per HTTP session
application	One per servlet context
websocket	One per WebSocket session

```
@Service
@Scope("prototype")
public class CartService {
    // fresh instance every injection
}
```

Default is singleton. Most beans should stay singleton. Use prototype for stateful objects that need to be fresh each time.

Configuration Approaches

Three ways to describe your dependency graph in Spring.

1. Annotations (most common)

```
@Service
public class OrderService {
    // ...
}
```

Spring scans the classpath and wires everything.

2. Java configuration

```
@Configuration
public class AppConfig {

    @Bean
    public EmailSender emailSender() {
        return new SmtplEmailSender("smtp.example.com");
    }

    @Bean
    public OrderRepository orderRepository(DataSource ds) {
        return new MySqlOrderRepository(ds);
    }
}
```

Useful when you need custom construction logic, or you're wiring third-party classes you can't annotate.

3. XML (legacy)

```
<bean id="orderService" class="com.example.OrderService">
    <constructor-arg ref="emailSender"/>
    <constructor-arg ref="orderRepository"/>
</bean>
```

Old projects still use it. Don't write new XML config unless you have a strong reason.

Best Practices

- **Prefer constructor injection.** Final fields, explicit dependencies, easy testing.
 - **Depend on interfaces, not concrete classes.** Easier to swap and mock.
 - **Keep constructors short.** A constructor with 8 parameters is telling you the class has too many responsibilities.
 - **Don't inject the container itself.** Asking for `ApplicationContext` to fetch beans manually defeats the point. (This is the Service Locator anti-pattern.)
 - **Configure once, inject everywhere.** Don't sprinkle wiring logic across services.
 - **Use `@Qualifier` when needed.** When two beans implement the same interface, be explicit about which one you want.
 - **Test without the container when you can.** Unit tests should just `new` the class with mocks. Save Spring tests for integration tests.
-

Common Anti-patterns

- **Service Locator.** Code that calls `ApplicationContext.getBean("foo")` instead of declaring a dependency. Hides the dependency from callers and breaks compile-time safety.
 - **Field injection.** Forces tests to use reflection or the framework. Skip it.
 - **Circular dependencies.** A depends on B, B depends on A. Usually a sign that responsibilities are split wrong. Refactor rather than bandaging it with setter injection.
 - **God beans.** A `@Service` with 15 dependencies. Split it up.
 - **New-ing inside a managed class.** Writing `new EmailSender()` inside a `@Service` bypasses DI entirely.
-

Common Gotchas

- **@Autowired on a final field doesn't work.** Field injection uses reflection, which can't write to `final`. Use constructor injection.
 - **Bean not found:** Spring couldn't find a candidate for the dependency. Check that the implementation is annotated and on the component scan path.
 - **Multiple beans of the same type:** Spring doesn't know which to inject. Add `@Qualifier` or mark one as `@Primary`.
 - **Circular dependency error:** A needs B, B needs A. Spring tries to break the cycle for setter injection but fails for constructor injection. Restructure or use `@Lazy`.
 - **Prototype injected into a singleton:** The prototype gets injected once and reused forever, defeating the scope. Use `@Lookup` or inject a `Provider<T>`.
 - **@Component not picked up:** The class is outside the component scan range. Move it or add it explicitly in config.
 - **Constructor injection without @Autowired** works since Spring 4.3 when there's only one constructor. With multiple constructors, you still need the annotation.
 - **Spring beans are eagerly created at startup.** A bug in a rarely-used bean still crashes the boot. Use `@Lazy` to defer.
-

Quick Reference

Concept	Key fact
Dependency Injection	Receive dependencies, don't create them
Inversion of Control	The framework calls your code
Constructor injection	Recommended default
Setter injection	For optional dependencies only
Field injection	Avoid in new code
DI Container	Builds the dependency graph automatically
<code>@Component</code> / <code>@Service</code>	Mark a class as managed by Spring
<code>@Autowired</code> / <code>@Inject</code>	Inject a dependency
<code>@Qualifier</code> / <code>@Named</code>	Disambiguate multiple candidates
<code>@Bean</code>	Factory method inside <code>@Configuration</code>
Bean scope	Singleton by default, prototype for fresh-each-time
Service Locator	Anti-pattern, hides dependencies